

DP w/ bitmasking
DP w/ Digits



Dynamic Programming Problem Solving 2

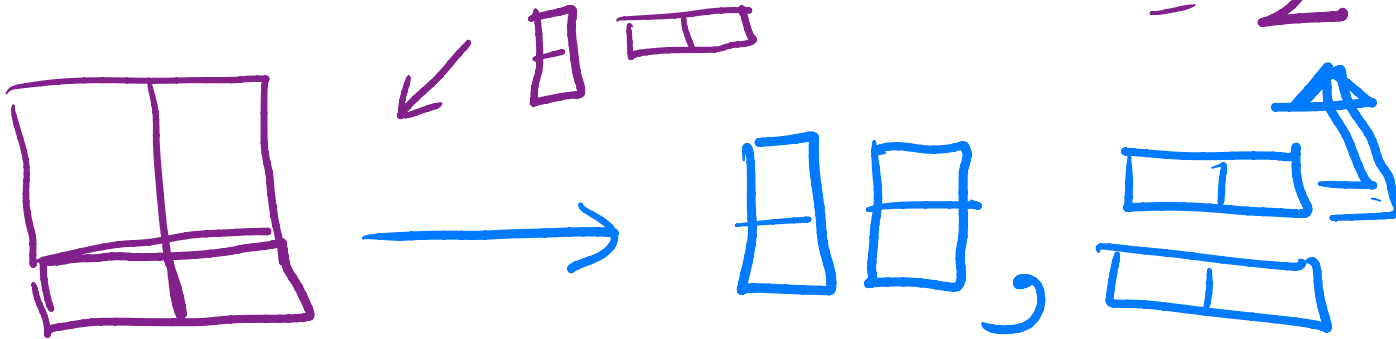
- Gaurish Baliga

Problem 1 [CSES]

Counting Tiles:

<https://cses.fi/problemset/task/2181>

$$n=2 \quad m=2$$



10 minutes

$$\underline{n \leq 10}$$

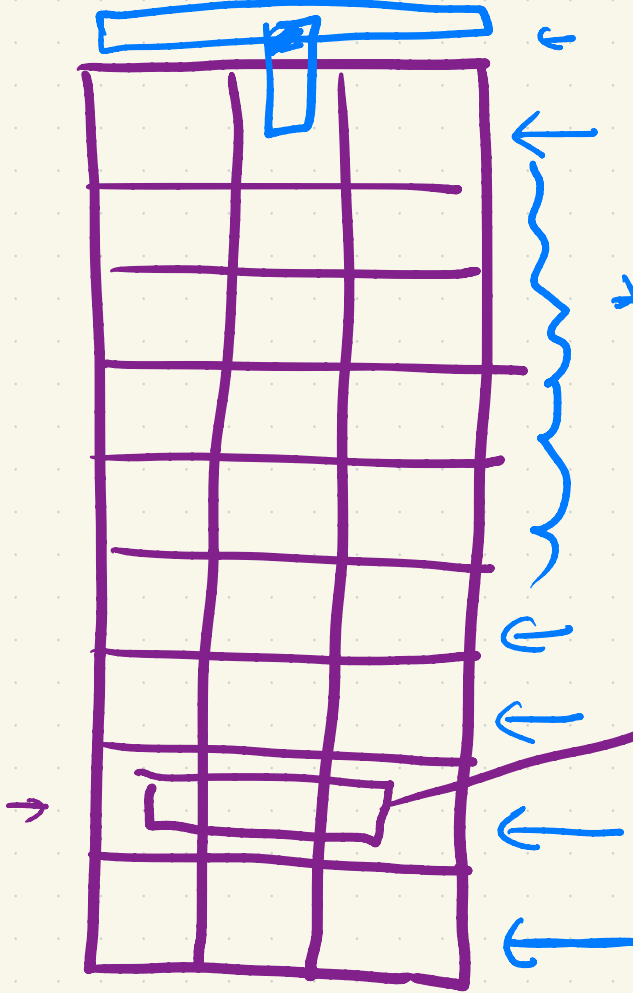


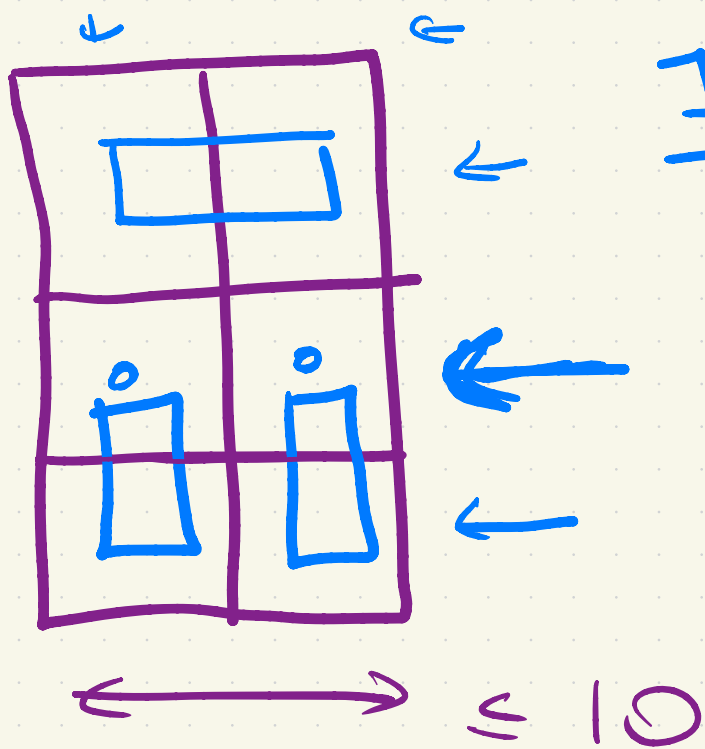
$$n=2 \quad m=2$$
$$= 2$$

$n \leq 10$

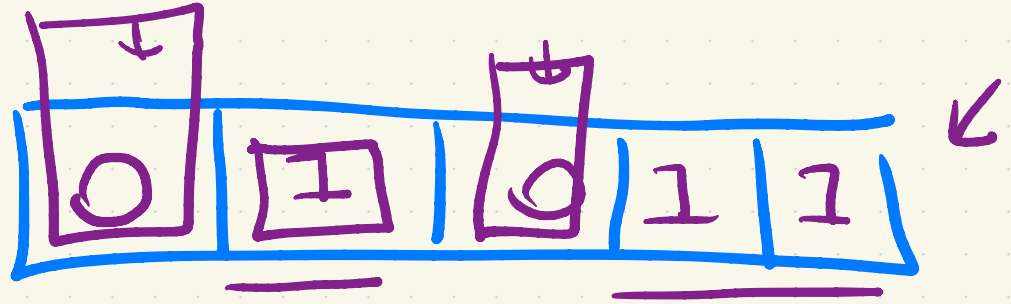
* You want to solve it
row by row

every row can be
represented as a
bitmask

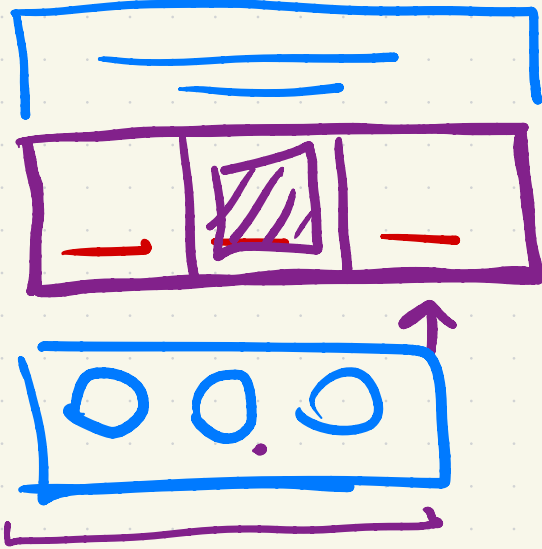




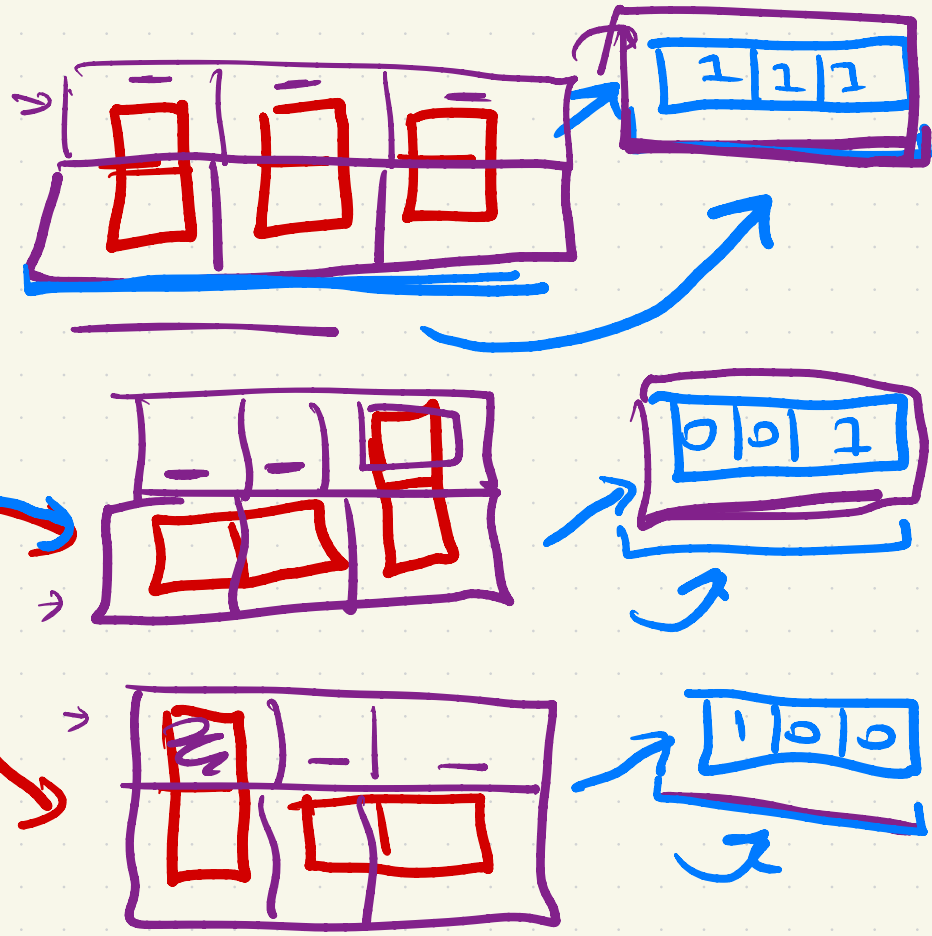
1 way to do so



transitions



transitions[mask]



transitions [000] =

{ 111, 100, 001 }

"overflows"

→

0	0	0	0
0	0	1	1

→

← next row

→

0	0	0
---	---	---

✓

$m+1$
 $m+1$

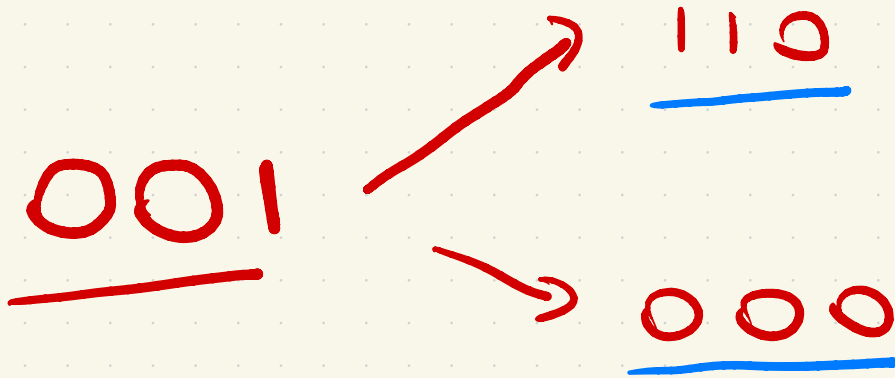
-	-	-
		1

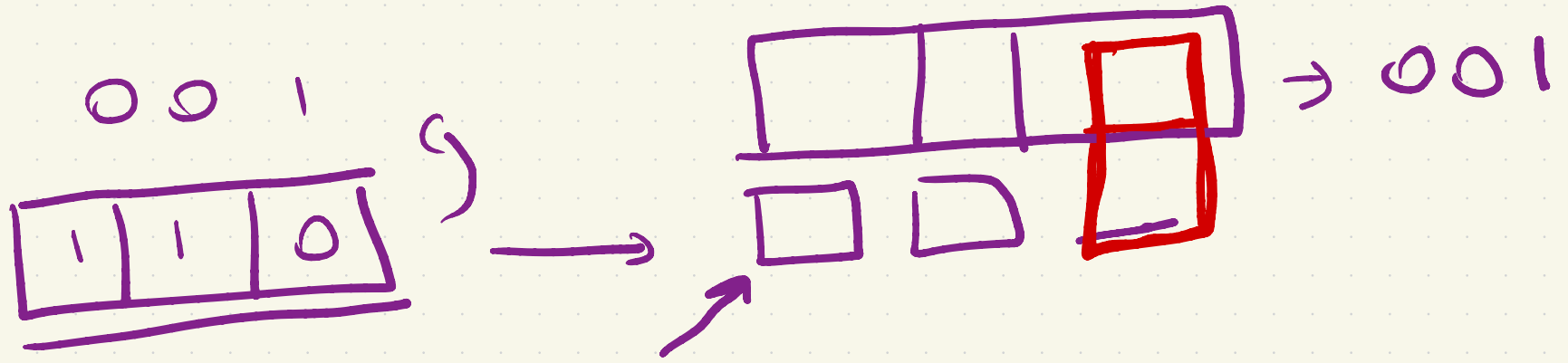
→

1	1	0
---	---	---

✗

$m+1$ ✗





$dp(i, \text{mask}) =$

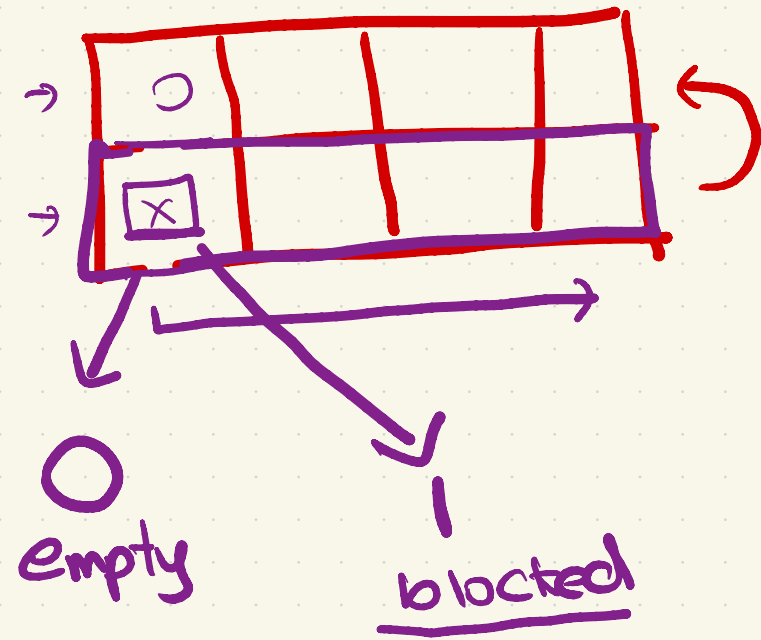
Magic Function



$\sum_{\substack{\text{newmask} \\ \text{c transitions}}} \underline{dp_{i+1, \text{newmask}}}$

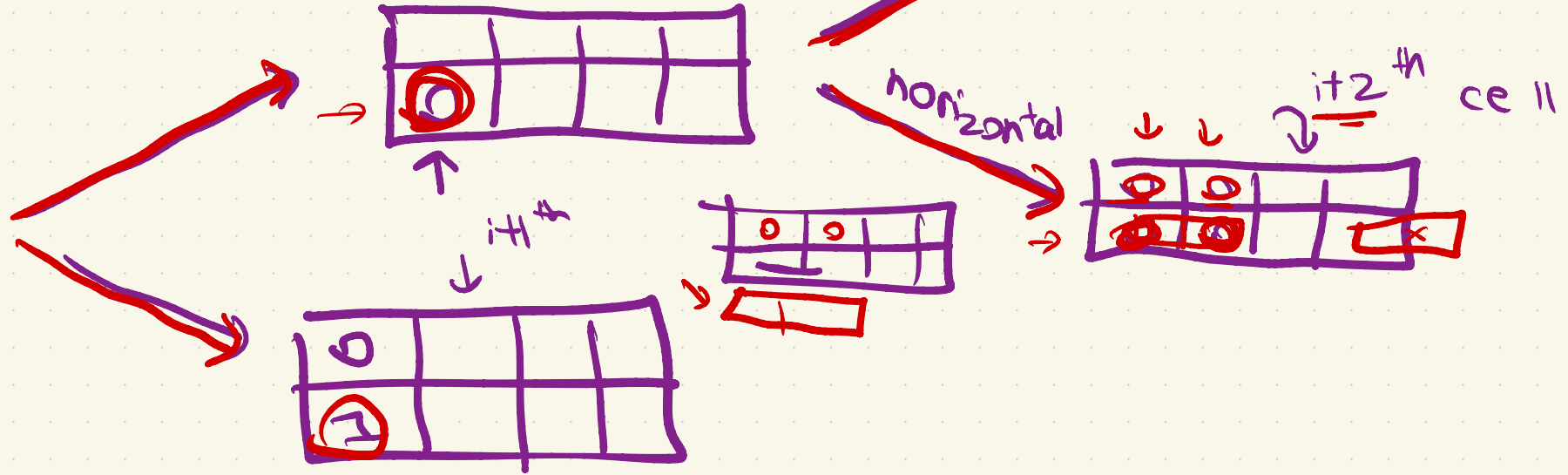
{ — — — — }

dp_i, mask $\rightarrow$ number of ways of
valid tilings ^{from i^{th} row to $m-1^{\text{th}}$ row} such that
we are on i^{th} row
and the state of
the i^{th} row is
represented with
mask



i^{th} cell
↓

mask 1, mask 2, i^{th} cell
current
next row



recursive



$$\approx O(3^n)$$

```
void generateTransitions(int index, int mask1, int mask2, int n, vector<vector<int>>& transitions) {  
    // If index exceeds n, stop (invalid path).  
    if (index > n) return;  
    // If we've exactly reached n, record the transition.  
    if (index == n) {  
        transitions[mask1].push_back(mask2);  
        return;  
    }  
    // Case 1: current block is "1" in mask1  
    generateTransitions(index + 1, mask1 | (1 << index), mask2, n, transitions);  
    // Case 2a: current block is "0" in mask1, and "1" in mask2  
    generateTransitions(index + 1, mask1, mask2 | (1 << index), n, transitions);  
    // Case 2b: skip 2 blocks (like placing a horizontal domino)  
    generateTransitions(index + 2, mask1, mask2, n, transitions);  
}
```

3 fn
calls

Total 3^n transitions

DP

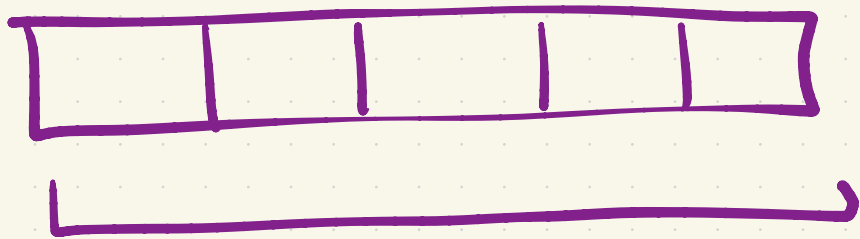


```
int f(int index, int mask, int n, int m, vector<vector<int>>& dp, const vector<vector<int>>& transitions) {  
    if (index == m) {  
        return (mask == 0 ? 1 : 0);  
    }  
  
    if (dp[index][mask] != -1) {  
        return dp[index][mask];  
    }  
  
    long long answer = 0;  
  
    for (auto newMask : transitions[mask]) {  
        answer = (answer + f(index + 1, newMask, n, m, dp, transitions)) % MOD;  
    }  
  
    return dp[index][mask] = answer;  
}
```

] mask should be empty

> memorization

* Main task



n

2^n

$arr[2^n]$

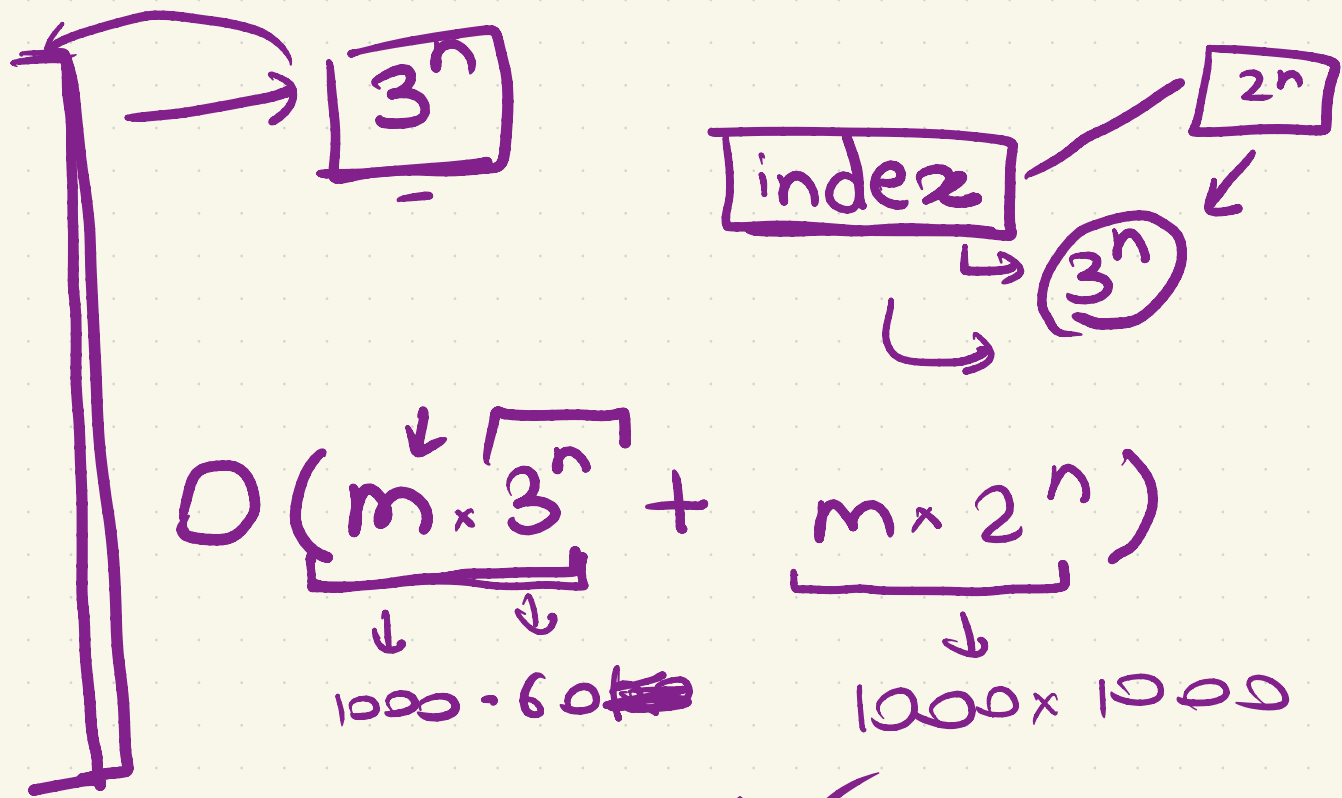
$mask = i$

$arr[i]$

$\{$

$\}$

000
010
.
.
.
.
.



$$O(\underbrace{m \times 3^n}_{1000 \cdot 6048} + \underbrace{m \times 2^n}_{1000 \times 1000})$$

$$6e7 + 1e6$$

$$\underbrace{1000 + 10001 \dots}_{\rightarrow m \times 3^n} \quad \underbrace{2^n \times 2^n \times 3^n}_{\rightarrow m \times 2^n \times 3^n}$$



$$(3^n)$$

$$m \times \boxed{3^n} \leftarrow \text{total no. of transition}$$

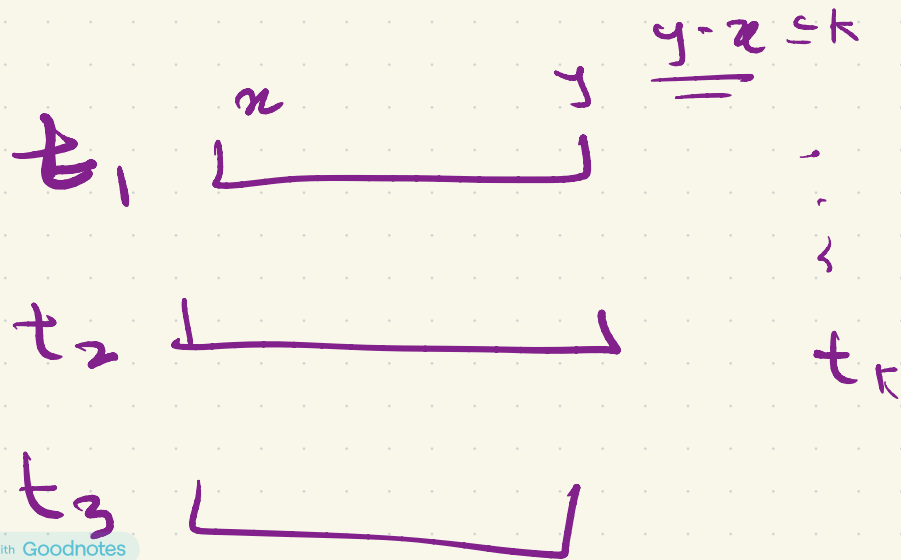
Problem 2



K Balanced Teams:

<https://codeforces.com/problemset/problem/1133/E>

$K \rightarrow$ cap on number of teams



maximise
the people
we chose

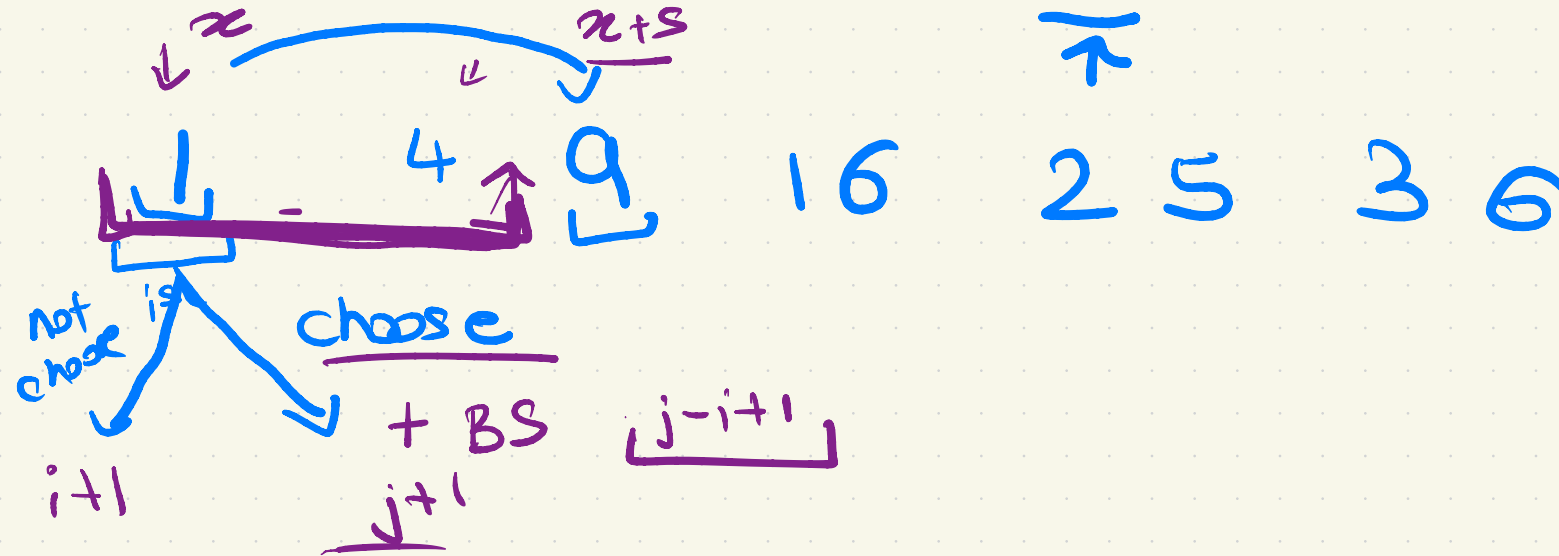
→ 1 1 2 3 4 4 5 7 7 7 8 8 $k=2$

→ Greedy won't work

Does sorting change anything?

Yes

$$K = 1$$



$dp(i, j)$

$\uparrow$

$[i \dots n-1]$

max people we
can accomodate

$\swarrow$ (we have picked up)
f teams yet.

Base Case $\rightarrow (j == \underline{k} \quad || \quad i = n) \text{ return } \underline{0};$
Case

$$\begin{array}{lcl}
 dp(i, j) & \xrightarrow[\substack{\text{choose} \\ a_k - a_i \leq s}]{\text{choose}} & \underbrace{k}_{a_k - a_i \leq s} C(k-i+1) + dp(k+1, \underline{j+1}) \\
 & \searrow \text{not choose} & \\
 & & dp(i+1, j)
 \end{array}$$

Final Subproblem

$$dp(\underbrace{0}_\uparrow, \underbrace{0}_{\uparrow 0 \text{ teams chosen}})$$

$$n, k \leq 5,000$$

3sec
↳ 1.5 sec For
Sure
TLE

$$O(nk \log n)$$

↳ Binary Search

$$\frac{5000 \times 5000 \times 12}{2.5 \times 10^7}$$

$$2.5 \times 10^7$$

↳ $> 1e8$
 $2e8$

TLE

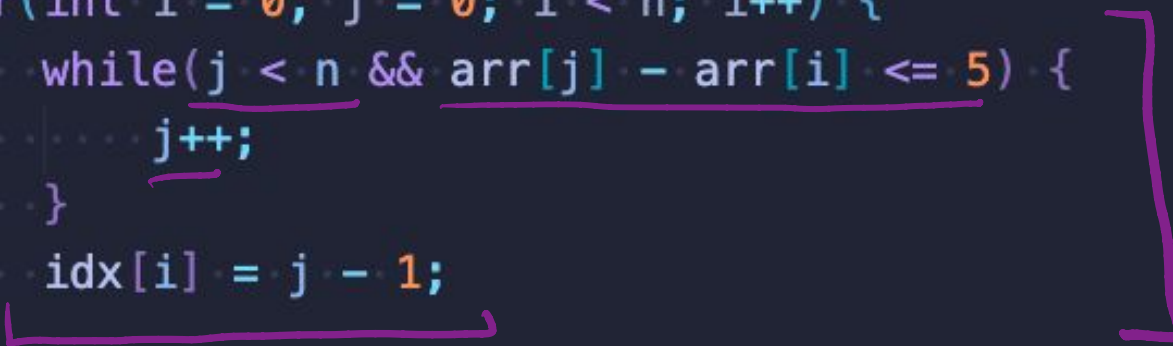
$O(nk)$ ✓ AC

ind[i]
↳ farthest k

index - precompute the value

→ 2 pointers → $O(n)$
→ Binary search → $O(n \log n)$

```
... vector<int>arr(n), idx(n);  
... for(auto &i : arr) cin >> i;  
  
... sort(arr.begin(), arr.end());  
  
... for(int i = 0, j = 0; i < n; i++) {  
...     while(j < n && arr[j] - arr[i] <= 5) {  
...         j++;  
...     }  
...     idx[i] = j - 1;  
... }
```



$O(nk)$ 2 dimensional DP problem

```
int f(int index, int teamsFormed, vector<vector<int>>&dp, vector<int>&idx, int n, int k){  
    if(index == n || teamsFormed == k) return 0;  
    * if(dp[index][teamsFormed] != -1) return dp[index][teamsFormed];  
  
    return dp[index][teamsFormed] =  
        max(f(index + 1, teamsFormed, dp, idx, n, k),  
            (idx[index] - index + 1) + f(idx[index] + 1, teamsFormed + 1, dp, idx, n, k));  
};
```

$k - i + 1$ $k + 1$

*

Problem 3

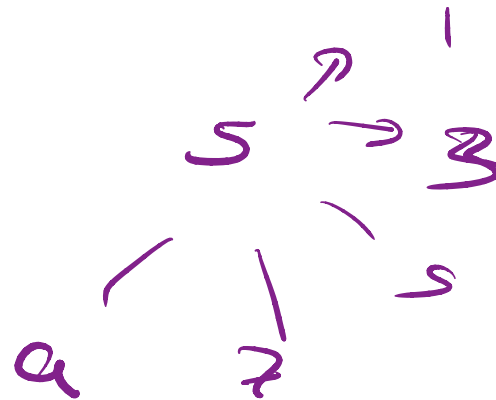
Digit DP last problem



Find how many numbers from L to R ($1 \leq L, R \leq 10^{18}$) have odd digits at odd indices and even digits at even indices

L R

[L, R]



$L=1$ $R=20$

$1, 20 \rightarrow 20$

1, 2, 3, 4, 5, 6, 7, 8,

9, 10, 11, 12, 13, 14, 15

16, 17, 18, 19, 20

Handwritten diagram illustrating the selection of the 1st and 2nd elements from a list [0, 0, 0, 1, 4, 3]. A red arrow points from the first three zeros to the '1', labeled '1st'. A purple arrow points from the '1' to the '4', labeled '2nd'.

$\boxed{i^{th}}$ $\rightarrow i \% 2 \rightarrow \text{odd}$

index % 2 index in number $\rightarrow$ $\rightarrow$ 2, 4, 6 ... $\rightarrow$ even
odd

Handwritten notes illustrating the parity of numbers:

- odd + odd = even
- odd + even = odd
- even + even = even

Examples of addition:

- odd + odd = even: $1 + 1 = 2$
- odd + even = odd: $1 + 2 = 3$
- even + even = even: $2 + 2 = 4$

$dp(\underline{\text{index}}, \underline{\text{tight}}, \underline{\text{start}}, \underline{\text{parity}})$

↳ number of numbers $\leq n$

$i = \text{index}$

is it bounded = tight

has number = start

parity of index = parity

$$|index| \times |tight| \times |start| \times |parity| \times 10$$

↓

$n = \text{size of } L/R$

$$n \times 2 \times 2 \times 2 \times 10$$

$$Ops = 80 \times \underline{n}$$

$$I \subseteq L \subseteq R \subseteq 10^{10^8}$$

$$|L| \subseteq |R| \subseteq 10^8$$

```

long long solveDP(int pos, int usedParity, bool start, bool tight) {
    if (pos == (int)digits.size()) {
        return start ? 1 : 0;
    }

    if (dp[pos][usedParity][start][tight] != -1) {
        return dp[pos][usedParity][start][tight];
    }

    long long ans = 0;

    int limit = tight ? digits[pos] : 9;

    for (int d = 0; d <= limit; d++) {
        bool nextTight = (tight && (d == limit));

        if (!start) {
            if (d == 0) {
                ans += solveDP(pos + 1, usedParity, false, nextTight);
            } else {
                bool expectingOdd = (usedParity == 1);
                bool digitIsOdd = (d % 2 == 1);
                if (digitIsOdd == expectingOdd) {
                    int nextUsedParity = usedParity ^ 1;
                    ans += solveDP(pos + 1, nextUsedParity, true, nextTight);
                }
            }
        } else {
            bool expectingOdd = (usedParity == 1);
            bool digitIsOdd = (d % 2 == 1);
            if (digitIsOdd == expectingOdd) {
                int nextUsedParity = usedParity ^ 1;
                ans += solveDP(pos + 1, nextUsedParity, true, nextTight);
            }
        }
    }

    dp[pos][usedParity][start][tight] = ans;
    return ans;
}

```

parity 3 odd
 seven
 memoisation
 bound
 odd
 start = 1
 start

Digit DP

Problem 4 (Homework)



Count the Number of Powerful Integers:

<https://leetcode.com/problems/count-the-number-of-powerful-integers/description/>

